

C++ Notes

Maria Gordenko, @mgordenko

2 сентября 2026 г.

This notes was written with great love for students, in the hope that it will help them learn such a challenging language as C++.

Thoughts from the Greats on Programming

"Premature optimization is the root of all evil in programming."

— Donald Knuth

"The first step in solving any problem is recognizing that it exists."

— Unknown Author

"C makes it easy to shoot yourself in the foot. C++ makes it harder, but when you do, it blows away your whole leg."

— Bjarne Stroustrup

Оглавление

1	Introduction to the Language. The First Program.	4
1.1	Running C++ Programs	4
1.2	Main elements of the C++ Language	5
1.3	The First Program	7
1.4	Program Structure	8
1.5	Printing Output to the Console	11
1.6	The Most Important Program	12
2	Fundamental Data Types	13
2.1	Data Types	13
2.2	Integer Data Types	13
2.3	Character Types	22
2.4	The Boolean Type	23
2.5	Floating-Point Numbers	24
2.6	The Empty Type: void	25
2.7	Stream Input	25
3	Variables	25
3.1	Memory Model	25
3.2	The C++ Memory Model	26
3.3	Variables	28
3.4	A Bit More About Expressions	36
3.5	The auto Keyword	41
4	Machine Arithmetic	41
4.1	Representing Data in a Computer	41
4.2	Representing Integers	42
4.3	Two's Complement	42
5	Bitwise Operations in C++	43
5.1	Bitwise Operators	43
6	Floating-Point Data Types (IEEE 754)	44
6.1	Representation of Floating-Point Numbers	45
6.2	Loss of Significance	46
6.3	Non-Associativity of Addition	47
7	Quirks of Working with Large Numbers	47
1	Seminars	49
1	Getting to Know C++. Data Types and Basic Operators.	50
2	Memory and Data Types. Storing Numbers in Memory. Quirks of Converting and Subtracting Large Numbers. Bitwise Operators.	56

1 Introduction to the Language. The First Program.

Historical Remark. The C++ language has a fairly rich history, starting with the C language, created in 1969 to simplify the implementation of low-level programs.

Here's roughly how it went:

- **C (1969/1972)** — the ancestor of C++. Created by Dennis Ritchie at Bell Labs. Designed for systems programming and implementing the UNIX operating system.
- **C with Classes (1979)** — Bjarne Stroustrup created a language that combined the speed of low-level languages with the convenience of developing in high-level ones. Classes, inheritance, and encapsulation were added.
- **C++ (1982/1983)** — the official name of the language. The name reflects its evolution from C ("++" is the increment operator in C). You can find a lot of memes about why they decided to use the postfix increment.
- **C++98 (1998)** — the first official C++ standard (ISO/IEC 14882:1998). The STL (Standard Template Library), namespaces, and RTTI were added.
- **C++11 (2011)** — also called "*the new C++*"; a so-called *major update* happened here, and a lot of what we'll be studying comes from it. Added: **auto**, lambda functions, range-based **for**, smart pointers, move semantics.
- **C++17 (2017)** — structured bindings, `std::optional`, `std::variant`, the filesystem library, parallel algorithms.
- **C++20 (2020)** — modules, concepts (concepts), coroutines, ranges, the spaceship operator (`<=>`).
- **C++23 (2023)** — one of the latest standards, and the one we'll be using, although C++26 already exists, but its changes aren't very significant for our basic course.

Modern C++ is fast and efficient, and **it's used wherever speed matters. You won't believe it**, but almost every application we work with is partly written in C++.

Well-known products written using C++ include *Windows*, *OS X*, *Google Chrome*, *Mozilla Firefox*, *YouTube*, *Photoshop*, *Telegram*, other programming languages ...

Important Remark.

Why do people choose C++:

- Maximum control over memory
- Predictable performance
- The ability to make low-level optimizations
- Compatibility with C and legacy code

Although this language is tricky, once you grasp its essence, there's no trickiness to it at all. Everything here is maximally logical.

1.1 Running C++ Programs

Definition 1.1. C++ is a **compiled** programming language. This means that the code (i.e. the program text) is first processed by a special program (**a compiler**), which

translates it into machine instructions and creates an executable file. This file can then be launched and run like any other program.

Examples of such languages are *C, C++, Basic, Go, Pascal, Rust, ...*

Definition 1.2. There are also **interpreted** programming languages. In these, the program text is fed as input to another program (an interpreter), which immediately begins executing the written code without a build step that produces an executable file.

Examples of such languages are *Python, PHP, Perl, Ruby*

Some languages (for example, *Java*) use a hybrid approach.

Any text file written according to the rules of the C++ language is C++ code.

To turn it into a program, it must be **compiled**, that is, the code must be passed to a compiler program.

1.2 Main elements of the C++ Language

The C++ language consists of a set of basic elements that are combined according to strict rules for building programs.

1.2.1 Alphabet

Definition 1.3. The **C++ alphabet** is the set of characters a C++ program can be made up of.

Any program in C++ consists of the following characters:

1. Digits (0, 1, ..., 9)
2. Letters of the Latin alphabet (a, b, ..., z, A, ..., Z)
3. Underscore _
4. Whitespace characters and line breaks
5. **Brackets:**
 - Curly braces: {} (code blocks, initialization)
 - Parentheses: () (function calls, expressions)
 - Square brackets: [] (arrays, lambda captures)
 - Angle brackets: <> (templates, comparison operators)
6. **Quotes and apostrophes:**
 - " — double quotes, used for strings: "Hello"
 - ' — single quotes, used for characters: 'A'

1.2.2 Keywords

Characters can form words, some of which have a special meaning.

Definition 1.4. Words with a special meaning are called **keywords**.

- String literals: `"a", "c", "n", "This is string"`
- Boolean constants — `true`, `false`
- The null pointer — `nullptr`

1.2.5 Operators

The language also includes operators.

Definition 1.7. An **operator** (or an operator in the mathematical sense) is a symbol or keyword that performs an operation on one or more operands. *Examples:* `+`, `-`, `*`, `/`, `=`, `==`, *etc.*

Each operator is characterized by its *arity*, *precedence*, and *associativity*.

Definition 1.8. **Arity** is the number of arguments an operator can take.

Definition 1.9. **Precedence** is a property that determines the order in which operations are evaluated.

Definition 1.10. **Associativity** is a property that determines the order in which operators with the same precedence are evaluated

 **Important!** The same operator symbol can have a different meaning, arity, and precedence depending on the number of operands and the data type.

Important Remark. In programming it is important to distinguish between the concepts of a **statement** and an **operator**.

- A **statement** is a complete programming-language command that performs some action. *Examples:* *a variable declaration, an assignment, a conditional statement, a loop, a return statement.*
- An **operator** (or an operator in the mathematical sense) is a symbol or keyword that performs an operation on one or more operands.

Example 0.1

```
10 / 3;    // Integer division - result 3
-10 / 3.;  // Floating-point division - result -3.3333
-5;        // Unary operator (negation)
5 - 3;     // Binary operator (subtraction)
```

You can read more about operators on [cppreference](#). We'll also touch on them when we discuss data types.

1.3 The First Program

It's time to talk about the first program. The minimal C++ program that does (almost) nothing looks like this:

```
int main() {}
```

This program consists of a keyword (**int**), an identifier (main), and brackets ((), {})

This code defines a function named main, which takes nothing (()), returns an integer (**int**), and does nothing ({}). A function with this name must be present in every program, since program execution begins with it.

By default, the main function returns 0. But this value can also be specified explicitly:

```
int main() {  
    return 0; // keyword return + the constant literal 0  
}
```

Also, we can perform integer arithmetic in C++ and return its result (though there isn't much point in doing so).

```
int main() {  
    return 2 + 2 + 2;  
}
```

Important Remark. First, the result of the expression to the right of **return** is evaluated, and then that value is returned from the main function.

Historical Remark. The main function in C++ returns an integer, which is interpreted by the operating system as the program's **exit code**.

The tradition of returning 0 on success comes from the UNIX operating system (1969), where a process could terminate successfully or with an error. This convention was inherited by C, and later by C++.

Why exactly 0?

1. There's a historical reason for this: in UNIX systems:

- 0 — successful completion (*success*)
- 1 or another nonzero value — an error (*failure*)

2. **The POSIX convention:**

- EXIT_SUCCESS = 0
- EXIT_FAILURE = 1

3. **The C++ standard (yes, it actually guarantees this):**

- If main doesn't return a value explicitly, the compiler automatically inserts **return 0**
- An explicit **return 0** is considered good style, so make use of it.

Important Remark.

- The C++ standard allows you to omit **return 0** in the main function. And yes, main is the only function where **return** can be omitted
- In other functions, the absence of a **return** leads to undefined behavior
- Some compilers require an explicit **return** even in main

1.4 Program Structure

As in other languages, C++ has a certain program structure, and it's important for us to follow it.

Definition 1.11. A **declaration** is the introduction (creation) of a named entity.

Example 0.2 Declaration of the function main:

```
int main(); // declaration
```

Notice that it appears here without an implementation; it's, in a sense, a declaration of intent to create such a function and use it in the future. This kind of declaration is a declaration without a definition.

In general, declarations only introduce new names but don't give them concrete meaning.

Definition 1.12. A **definition** is a declaration with complete information about the entity being created.

Example 0.3 Example of a function definition:

```
int main(); // declaration
int main() { // definition
    return 0;
}
```

 **Important!** A program must have exactly one definition of the main function. And main cannot be left undefined.

Definition 1.13. **Preprocessor directives** are auxiliary instructions that begin with # and perform manipulations on the program text.

Example 0.4 Examples of preprocessor directives:

```
// Insert the file "file" from the current folder
#include "file"

// Insert the file "file" from the system paths
#include <file>

// Replace X with Y throughout the code (macro substitution)
#define X Y
```

Functions (in particular, the main function) describe a sequence of actions that are executed in order, from top to bottom.

 **Important!** Recall the definition of a statement — you'll need it again later.

At this point we're familiar with one statement — the function-return statement **return**.

Some actions (statements) require computations to be performed. In C++, computations are handled by *expressions*.

Definition 1.14. An **expression** is a sequence of operators and their operands that specifies a computation.

Every expression has a result (which may be empty), which is *substituted* in its place.

Example 0.5

```
int main() {  
    return 2 + 2 * 2; // 2 + 2 * 2 - an expression with the  
                       result 6.  
}
```

Sometimes you want to leave a note or a reminder in the code about what a particular block of code means. But we know you can't just write plain text into the code, since it will most likely not satisfy the grammar rules of C++. For this purpose, the language has *comments*.

Definition 1.15. A **comment** is a piece of code text that is ignored when the program is built.

There are two types of comments in C++:

Example 0.6

```
// single-line comment  
/* multi-line  
comment */
```

Comment. Even though we're talking about C++, it's worth noting that Java has an interesting quirk related to processing Unicode characters *before* comments are parsed. This makes it possible to create so-called *executable comments*:

```
public static void main(String... args) {  
    // the comment below is not a typo!  
    // \u000d System.out.println("Hello World!");  
}
```

In this example, the sequence `\u000d` represents the Unicode *carriage return* line-break character. The Java compiler processes Unicode escapes **before** determining what's inside a comment. So the code effectively becomes:

```
public static void main(String... args) {  
    // the comment below is not a typo!  
    //  
    System.out.println("Hello World!");  
}
```

In other words, here's what happens:

1. The Java compiler first replaces all Unicode escapes (e.g., `\u000d`, `\u000a`)
2. Only after that does it parse the code's structure, including comments
3. As a result, the line-break character effectively falls outside the comment
4. The next line becomes executable code

 **Important!** This is a Java-specific quirk and doesn't work in C++. In C++ comments are *fully* ignored, and Unicode characters inside them are not processed as escape sequences.

Exercise 1.2 Look at the code and say where in it you can find:

1. Declarations?
2. Definitions?
3. Directives?
4. Statements?
5. Expressions?
6. Comments?

```
#define TWO 2

// a primitive C++ program

int main();

int main() {
    return TWO + TWO * TWO;
}
```

1.5 Printing Output to the Console

All the programs we wrote earlier returned their result to the operating system as an *error code*:

```
int main() {
    return 2 + 2 * 2;
}
```

This is quite inconvenient, because

1. A program's result isn't always an integer
2. Error codes can only take values from 0 to 255
3. The result has to be retrieved separately using a special environment variable — `echo $?`

To print a result to the screen, C++ typically uses *stream output*:

```
#include <iostream> // 1
int main() {
    std::cout << 2 + 2 * 2; // 4
    return 0; // 0 - no errors, success
}
```

- line 1: We know that **#include** is a directive meaning the *insertion* of another file into our program's text. The `iostream` (Input-Output STREAM) file contains everything needed for working with output.
- line 4: This line contains the **expression** that outputs the result of `2 + 2 * 2` to the `std::cout` (Console OUTput) stream.

Other kinds of information can also be written to the output stream, such as strings. It's good style to output a line break at the end of the result (for a nicer-looking message on screen). This is handled by the special character `'\n'`.

Example 0.7

```
#include <iostream>
int main() {
    std::cout << "2 + 2 * 2 = ";
    std::cout << 2 + 2 * 2;
    std::cout << '\n';
    return 0;
}
```

All the output can be combined into a single expression:

```
#include <iostream>
int main() {
    std::cout << "2 + 2 * 2 = " << 2 + 2 * 2 << '\n';
    return 0;
}
```

1.6 The Most Important Program

Using what we've learned so far, let's write a program that greets the world:

```
#include <iostream>
int main() {
    std::cout << "Hello, world!\n";
    return 0;
}
```

Important Remark.

`\n` and `std::endl` are both used to output a newline in C++, but they work differently.

Let's look at `\n` (the newline character):

- A character constant (ASCII value 10)
- Only adds a newline character to the output stream
- Faster, no extra operations

And here's what we know about `std::endl` (a manipulator):

- A stream manipulator from the `<iostream>` library
- And it works like this:
 1. Adds `\n` to the stream
 2. Calls `std::flush()` to flush the buffer
- Slower, due to the buffer flush

When to use which

- Use `\n`:
 - Whenever you don't need an immediate buffer flush
 - In loops with a large amount of output
 - In high-performance code
 - For a regular line break
- Use `std::endl`:
 - When you need immediate output (for example, a progress indicator)
 - When writing to log files
 - To guarantee that a message appears right away

Historical Remark. The concept of buffering comes from the C language and from operating systems. Flushing the buffer guarantees that the data has actually been written to the output device (screen, file, network).

2 Fundamental Data Types

2.1 Data Types

Definition 2.1. A *data type* is a property of program entities (we won't say strictly that it's only for variables here) that determines:

1. The set of allowed values
2. The allowed operations
3. The low-level representation (how the bits are interpreted, i.e. **how** to read that number)

C++ has a large number of data types. Moreover, you can (*and we will*) create your own types as well.

However, every data type is composed of, or derived from, one of the fundamental types:

- The *empty/void* type — **void**
- Integer types — **int**, **short**, **long**, **long long**
- Floating-point numbers — **float**, **double**, **long double**
- Character types — **char**, **char8_t**, **char16_t**, **char32_t**
- The boolean type — **bool**
- The null pointer — **std::nullptr_t**

2.2 Integer Data Types

The most commonly used data type is the integer type. So we'll devote the most attention to it (and not only this time).

2.2.1 int

The primary integer type is `int`.

Important Remark. The C++ standard guarantees that the size of `int` is **at least** 16 bits (2 bytes).

In practice, as a rule (almost always), `int` occupies 32 bits (4 bytes), i.e. it takes values in the range

$$[-2\,147\,483\,648; 2\,147\,483\,647]$$

or, in a more convenient form:

$$[-2^{31}; 2^{31} - 1]$$

Literals of type `int` include — 0, 1, -4, 100, 576843, etc.

Exercise 2.1 Based on your knowledge of the size of the `int` data type, show how to compute its range. After all, obviously you should never memorize it.

2.2.2 long

Important Remark. The C++ standard guarantees that the size of `long` is **at least** 32 bits (4 bytes) and **no smaller** than the size of `int`.

This means that values of type `long` can lie within the range

$$[-2\,147\,483\,648; 2\,147\,483\,647]$$

or, in a more convenient form:

$$[-2^{31}; 2^{31} - 1]$$

Literals of type `long` include: 0L, 1L, -4L, 100L, 1123576843L.

Important Remark. How does long differ from int and why is it needed?

Although on many modern platforms the types `int` and `long` have the same size (usually 32 bits), this is historical. On older 16-bit systems, where `int` was 16 bits, `long` was used for 32-bit values. Nowadays it's kept for backward compatibility, and to guarantee having at least a 32-bit number.

Exercise 2.2 Based on your knowledge of the size of the `long` data type, show how to compute its range. After all, obviously you should never memorize it.

2.2.3 long long

Important Remark. The C++ standard guarantees that the size of **long long** is **at least** 64 bits (8 bytes) and **no smaller** than the size of **long**.

This means that values of type **long long** can lie within the range

$$[-9\,223\,372\,036\,854\,775\,808; 9\,223\,372\,036\,854\,775\,807]$$

or, in a more convenient form:

$$[-2^{63}; 2^{63} - 1]$$

Literals of type **long long** include `0LL`, `1LL`, `-4LL`, `100LL`, `1123576843LL`

Exercise 2.3 Based on your knowledge of the size of the **long long** data type, show how to compute its range. After all, obviously you should never memorize it.

2.2.4 short

Important Remark. The C++ standard guarantees that the size of **short** is **at least** 16 bits (2 bytes) and **no larger** than the size of **int**.

This means that values of type **short** can lie within the range

$$[-32\,768; 32\,767]$$

or, in a more convenient form:

$$[-2^{15}; 2^{15} - 1]$$

The type **short** has no literals of its own.

Important Remark. The size of a type can be found using the **sizeof** operator:

- **sizeof**(type) — the size of the type in bytes
- **sizeof**(expression) — the size of the expression's type
- **sizeof** variable — the size of a variable (the parentheses can be omitted)

```
#include <iostream>

int main() {
    std::cout << "sizeof(int) = " << sizeof(int) << '\n';
    std::cout << "sizeof(short) = " << sizeof(short) << '\n';
    std::cout << "sizeof(long) = " << sizeof(long) << '\n';

    int a;
    std::cout << "sizeof(a) = " << sizeof(a) << '\n';
    std::cout << "sizeof a = " << sizeof a << '\n'; // the
```

```
parentheses can be omitted  
    return 0;  
}
```

Important Remark. To summarize, the C++ standard guarantees the following relationship between integer sizes:

$$\text{sizeof(char)} \leq \text{sizeof(short)} \leq \text{sizeof(int)} \leq \text{sizeof(long)} \leq \text{sizeof(longlong)}$$

2.2.5 The signed/unsigned Modifiers

By default, all integer types are *signed*, meaning they can store both positive and negative numbers. To emphasize this, the **signed** modifier can be added to the type name (though this is not required at all):

```
int == signed int == signed  
long == signed long == long int == signed long int  
long long == signed long long == long long int == signed long long  
    int  
short == signed short == short int == signed short int
```

If a value is meant to hold only non-negative numbers, you can add **unsigned** to the type name.

In this case, the allowed values change as follows:

- 2 bytes — [0; 65'535]
- 4 bytes — [0; 4'294'967'295]
- 8 bytes — [0; 18'446'744'073'709'551'615]

```
unsigned int == unsigned  
unsigned long == unsigned long int  
unsigned long long == unsigned long long int  
unsigned short == unsigned short int
```

Unsigned literals — 1u, 4ul, 6ull

2.2.6 Operations on Integers

All arithmetic operations can be performed on integers (except **short**!). The rules for these operations are as follows:

- The result of an operation has the same type as its operands
- When unsigned numbers overflow, arithmetic is performed modulo the size of that type
- In every other case, overflow is Undefined Behaviour (UB)
- Division by 0 is Undefined Behaviour
- Bitwise shifts of negative numbers can also lead to UB

If, however, the operands have different types (or one is **short**), then:

- (**signed** / **unsigned**) **short** is converted to **int**
- The narrower type is converted to the wider one
- The signed type is converted to the unsigned one

Example 0.8

```
0 + 0l; // long
0ll + 0; // long long
0l + 0ll; // long long
0u + 0; // unsigned int
0 + 0ul; // unsigned long
0ul + 0ll; // unsigned or signed long long
```

2.2.7 Fixed-Width Integer Types

On different systems, integer types can have different sizes, which complicates things.

To solve this problem, *fixed-width* types were introduced back in C++11:

They have **exactly** the size stated in their name. To use them, you need to include the header file `<cstdint>`

Example 0.9

```
#include <cstdint>

int main() {
    // Signed fixed-width types
    int8_t a = 127;    // Exact size: 1 byte
    int16_t b = 32767; // Exact size: 2 bytes
    int32_t c = 100;   // Exact size: 4 bytes
    int64_t d = 1000;  // Exact size: 8 bytes

    // Unsigned fixed-width types
    uint8_t e = 255;    // 0..255
    uint16_t f = 65535; // 0..65535
    uint32_t g = 100;   // 0..4294967295
    uint64_t h = 1000;  // 0..18446744073709551615

    return 0;
}
```

Historical Remark. You may encounter two different naming conventions across systems and libraries:

- `uint8_t`, `uint16_t`, `uint32_t`, `uint64_t` — the **standard names** from the `<cstdint>` header (C++11) or `<stdint.h>` (C99)
- `u_int8_t`, `u_int16_t`, `u_int32_t`, `u_int64_t` — **non-standard names** found in some UNIX systems (for example, older versions of BSD) and libraries

 **Important!** Always use the standard names `uint8_t` etc. from `<cstdint>`. Non-

standard underscored names may not be supported on all platforms and don't guarantee portability (cross-platform compatibility).

2.2.8 Basic Arithmetic Operations on Integers

C++ provides both basic arithmetic operators and compound assignment operators, which combine an operation with an assignment.

	Description
=	Assignment
+	Addition, unary +
-	Subtraction, unary -
*	Multiplication
/	Division (integer)
%	Remainder
++	Increment (add 1)
--	Decrement (subtract 1)

2.2.9 Compound Assignment Operators

	Description
+=	Addition assignment
-=	Subtraction assignment
*=	Multiplication assignment
/=	Division assignment
%=	Remainder assignment

Example 0.10

```
#include <iostream>

int main() {
    int a = 10, b = 3;

    int sum = a + b;      // 13
    int diff = a - b;     // 7
    int prod = a * b;     // 30
    int quot = a / b;     // 3 (integer division)
    int rem = a % b;      // 1 (remainder)

    a += 5;               // a = a + 5 -> 15
    a -= 2;               // a = a - 2 -> 13
    a *= 2;               // a = a * 2 -> 26
    a /= 4;               // a = a / 4 -> 6
```

```
a %= 4;    // a = a % 4 -> 2

int x = 5;
x++;       // x = 6 (postfix)
++x;       // x = 7 (prefix)
x--;       // x = 6 (postfix)
--x;       // x = 5 (prefix)

int y = +10; // +10 (explicitly positive)
int z = -10; // -10 (negative)

return 0;
}
```

2.2.10 Quirks of Integer Arithmetic

Important Remark.

- **Integer division:** $10 / 3 = 3$ (the fractional part is discarded)
- **Remainder:** $10 \% 3 = 1$
- **Division by zero:** leads to undefined behavior (UB)
- **Overflow:** for signed types — UB; for unsigned types — modular arithmetic.

Example 0.11

```
#include <iostream>

int main() {
    std::cout << "10 / 3 = " << 10 / 3 << '\n';    // 3
    std::cout << "-10 / 3 = " << -10 / 3 << '\n';    // -3

    std::cout << "10 % 3 = " << 10 % 3 << '\n';    // 1
    std::cout << "-10 % 3 = " << -10 % 3 << '\n';    // -1
    std::cout << "10 % -3 = " << 10 % -3 << '\n';    // 1

    int x = 10;
    x += 2 * 3;    // x = x + (2 * 3) = 16
    x -= 5 / 2;    // x = x - (5 / 2) = 14
    x %= 4 + 1;    // x = x % (4 + 1) = 4

    int a = 5;
    int b = a++ * 2;    // b = 10, a = 6 (postfix)
    int c = ++a * 2;    // c = 14, a = 7 (prefix)

    return 0;
}
```

2.2.11 Bitwise Logical Operations

Bitwise operations act directly on the bits of numbers, which makes it possible to work efficiently with flags and masks and to perform low-level optimizations.

	Description
&	Bitwise conjunction (AND)
	Bitwise disjunction (OR)
^	Bitwise exclusive OR (XOR)
~	Bitwise negation (NOT)
<<	Left shift
>>	Right shift

Example 0.12

```
#include <iostream>

int main() {
    int a = 0b1010; // 10 in binary
    int b = 0b1100; // 12 in binary

    // Bitwise conjunction (AND)
    std::cout << " (" << (a & b) << ")\n"; // 1000 (8)

    // Bitwise disjunction (OR)
    std::cout << " (" << (a | b) << ")\n"; // 1110 (14)

    // Exclusive OR (XOR)
    std::cout << " (" << (a ^ b) << ")\n"; // 0110 (6)

    // Bitwise inversion (NOT)
    std::cout << " (" << (~a) << ")\n"; // depends on the
    size of int

    // Shifts
    std::cout << " (" << (a << 1) << ")\n"; // 10100 (20)
    std::cout << " (" << (a >> 1) << ")\n"; // 0101 (5)

    return 0;
}
```

2.2.12 Comparison Operators

Comparison operators are used to compare two values and return a boolean value, **true** or **false** (we'll see this again later as the **bool** type).

	Description
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Example 0.13

```

#include <iostream>

int main() {
    int a = 10, b = 5, c = 10;

    // Equality
    std::cout << std::boolalpha;
    std::cout << a << " == " << c << " : " << (a == c) <<
'\n';    // true
    std::cout << a << " == " << b << " : " << (a == b) <<
'\n';    // false

    // Inequality
    std::cout << a << " != " << b << " : " << (a != b) <<
'\n';    // true
    std::cout << a << " != " << c << " : " << (a != c) <<
'\n';    // false

    // Greater/less than
    std::cout << a << " > " << b << " : " << (a > b) << '\n';
    // true
    std::cout << b << " < " << a << " : " << (b < a) << '\n';
    // true
    std::cout << a << " > " << c << " : " << (a > c) << '\n';
    // false

    // Greater/less than or equal
    std::cout << a << " >= " << c << " : " << (a >= c) <<
'\n';    // true
    std::cout << a << " <= " << c << " : " << (a <= c) <<
'\n';    // true
    std::cout << b << " >= " << a << " : " << (b >= a) <<
'\n';    // false

    // Chained comparisons
    bool in_range = (5 < a) && (a < 15);

```

```
std::cout << "5 < " << a << " < 15 : " << in_range <<
'\n'; // true

return 0;
}
```

Important Remark.

- The result of a comparison has type **bool**
- Comparing floating-point numbers can be imprecise due to rounding errors
- In C++ you cannot chain comparisons the way you would in math ($a < b < c$)
- Comparing pointers uses this same set of operators

2.3 Character Types

Character data types are used both on their own and together with strings (which will be covered later in the course).

2.3.1 char

The **char** type is used to store characters.

A character is represented by a 1-byte integer, according to the [ASCII table](#)

Important Remark. Size of char The type **char** always has a size of **exactly 1 byte** (usually 8 bits). However, the C++ standard allows a byte to have more than 8 bits (for example, on some embedded systems).

Character literals — `'a'`, `'1'`, `'n'`, `'@'`, ...

C++ has three distinct **char** types: Which type is used by default depends on the

Type	Description
char	May be signed or unsigned depending on the compiler and platform. Always 1 byte.
signed char	Always signed. Range: -128 to 127. Used for small integers.
unsigned char	Always unsigned. Range: 0 to 255. Used for byte data.

implementation.

2.3.2 char_t, char16_t, char32_t

Wide character types are used to store characters from the *UTF – 16* and *UTF – 32* encodings. You can read more via this [link](#).

2.3.3 Arithmetic

Since characters are represented by integers, the char type can be treated as an 8-bit integer, and all arithmetic operators can be used with it.

```
'E' + ('a' - 'A') == 'e'
```

Notation. When performing arithmetic, numbers whose type has a rank lower than **int** are converted to **int**.

Example 0.14

```
'a' + 0; // int
'a' + 0l; // long
'a' + 0ll; // long long
'a' + 'a'; // int
```

2.4 The Boolean Type

2.4.1 bool

Objects of the boolean type can take just 2 values: **true** / **false**.

It has a size of 1 byte.

It's usually used to store the result of a comparison:

Example 0.15

```
5 > 6; // false
5 < 6; // true
5 >= 6; // false
5 <= 6; // true
5 == 6; // false
5 != 6; // true
```

Objects of type **bool** can serve as operands of logical operators:

Example 0.16

```
5 > 6 && 5 < 6; // false (logical "and")
5 > 6 || 5 < 6; // true (logical "or")
!(5 > 6); // true (logical "not")
```

 **Important!** C++ also allows using the special words **and**, **or**, **not**. But this is considered poor style, since these keywords come from the C language, which originally lacked the special symbols. Using them is not recommended, and on our course it is forbidden.

Also, **&&** and **||** are special operators (lazy):

1. It's guaranteed that the expression on the left is evaluated before the one on the right.
2. If the left-hand value is **true**, the right-hand side of **||** will not be evaluated.

3. If the left-hand value is **false**, the right-hand side of **&&** will not be evaluated.

Example 0.17

```
5 < 6 || ... ; // the right-hand side is not evaluated
                (short-circuit, left true)
5 > 6 && ...; // the right-hand side is not evaluated
                (short-circuit, left false)
```

2.4.2 Arithmetic

The boolean type in C++ is a kind of integer type and can be used in arithmetic expressions. Here **true** == 1, and **false** == 0:

Example 0.18

```
true + 5; // int: 6
10l * false; // long: 0
```

The reverse is also true: when substituted into a logical operator, a nonzero value is interpreted as **true**, and zero as **false**:

```
5 && 1; // true
0 || 0; // false
!-1;    // false
```

2.5 Floating-Point Numbers

Floating-point numbers are used to store rational numbers.

float — single-precision numbers (4 bytes, roughly $[\pm 10^{-38}; \pm 10^{38}]$)

double — double-precision numbers (8 bytes, roughly $[\pm 10^{-308}; \pm 10^{308}]$)

long double — extended-precision numbers (12 or 16 bytes, roughly $[\pm 10^{-4932}; \pm 10^{4932}]$)

Example 0.19 Floating-point literals:

```
0., 1.5, 3.14159; // double
0f, 1.5F, 3.14159f; // float
0l, 1.5L, 3.14159l; // long double
123.456e10; // 123.456 * 10^10
123.456e-10; // 123.456 * 10^(-10)
```

Floating-point numbers have their own quirks:

- The same arithmetic operators can be applied as with integers (except for bitwise operators and the remainder): $0.1 + 0.2 * 5.67 / 0.9$;
- Keep in mind that fractional numbers have limited precision in computations: $0.1 + 0.2 \neq 0.3$;
- Floating-point numbers are always signed.
- There are special values: **+inf**, **-inf**, **nan**:

Example 0.20

```
1. / 0; //inf
-1. / 0; //-inf
0. / 0; //nan
```

- If the floating-point types don't match, the narrower argument is converted to the wider one:

Example 0.21

```
5.0 + 1.5f; // double
5.0 + 1.5l; // long double
```

- When performing an arithmetic operation on an integer and a fractional number, the integer is converted to the fractional type: `1 + .0f; // float`

2.6 The Empty Type: void

The type **void** is a type with an empty set of values.

Its main use is to indicate that an expression returns nothing (there is no result). We'll get to know it in more detail in the context of functions.

2.7 Stream Input

We almost forgot — how do you read data from the console? To read values from a stream, you can use the `<iostream>` library:

Example 0.22

```
#include <iostream>
int main() {
    int x;
    int y;
    std::cin >> x >> y;
    std::cout << x + y << '\n';
    return 0;
}
```

This program reads 2 integers from the console and prints their sum.

3 Variables

3.1 Memory Model

The computer's memory has a fairly complex technical structure. Moreover, memory management methods can vary greatly depending on the architecture and OS being

used.

Memory models provide a convenient abstraction to work with, which in most situations lets you write code without thinking about the low-level details of how interactions are implemented.

When writing code, we're guided by the provided memory model, while the implementation and execution of specific instructions rests on the compiler's shoulders.

3.2 The C++ Memory Model

Let's assume the following:

- The C++ memory model is a *sequence of numbered memory cells*, which are called *bytes*.
- The number of a memory cell is called its *address*.
- Thus, a *byte* is the minimum addressable unit of memory.
- The C++ standard guarantees that the size of a byte exactly matches the size of objects of type *char*.
- All memory cells are equal. With the exception of the zero cell: nothing can be written there, and nothing can be read from it.
- Sequences of cells can have different interpretations depending on which type of element we're currently considering.

Important Remark. Why does the type `bool` take up 1 byte?

Although a single bit (0 or 1) is enough to store a boolean value, the type `bool` in C++ takes up at least 1 byte, and here's why:

- Processors typically work with bytes as the minimum addressable unit of memory. Accessing individual bits requires extra operations (reading a byte, shifting).
- To optimize memory access, compilers align data on boundaries that are multiples of the machine word size. 1 byte is the minimum alignment.
- Working with whole bytes/words is faster than working with individual bits. Modern processors are optimized for byte-level operations.
- **And also, believe it or not, compatibility is to blame!** The C language didn't have a `bool` type before the C99 standard, and boolean values were represented as `int` (0 — false, nonzero — true). C++ preserved this compatibility.

If you're curious to try alignment in action right now — try running this code:

Example 0.23

```
struct A1 {
    char a;
    int b;
    long c;
};

struct A2 {
    char a;
    long c; // just swapped the fields around
    int b;
};

int main() {
    std::cout << sizeof(A1) << '\n';
    std::cout << sizeof(A2) << '\n';
}
```

The result will be 16 and 24.

This is *alignment*.

The processor prefers to read data not from just anywhere, but from addresses that are multiples of a power of two:

- int — at an address that's a multiple of 4,
- long, double — at an address that's a multiple of 8,
- SIMD types — at 16, 32, or even 64.

To follow these rules, the compiler *inserts "holes" (padding)* between the fields of a structure so that everything works quickly.

In A1, the fields go "from small to large," and there's almost no padding. In A2, long immediately follows char — and the compiler is forced to insert **7 bytes of emptiness** so that long starts at an address that's a multiple of 8. And at the end there are **4 more bytes** — so that arrays of such structures don't break the alignment of the next element.

If you want to remove alignment, you can insert these lines at the top of the code.

```
#pragma pack(push, 1)
// ... structures ...
#pragma pack(pop)
```

Now the answer will be 13 and 13 — there's no alignment at all. But: **don't do this without a good reason**. On some architectures the program will crash. Even on x86 it will be slower.

Important Remark. Why does alignment exist in the first place?

Memory is read in *blocks* — most often **8 bytes** (64 bits) at a time. Imagine: an `int` sits at address 62 (let's use decimal for simplicity). Then its bytes span two adjacent blocks:

- the first byte — in the block 56-63,
- the last — already in 64-71.

To read such an `int`, the processor needs to:

1. load two blocks,
2. cut out the needed 4 bytes,
3. assemble them.

This is **slow**, **costly**, and **unnecessary** if the data can simply be placed at an even address.

Hence: **the address of a variable of type `T` must be divisible by `alignof(T)`** (an operator that returns the required alignment of type `T` in bytes.).

3.3 Variables

Definition 3.1. A **variable** is a named region of memory.

To create a variable, you need to specify its type, give it a name, and, optionally, an initial value:

Example 0.24

```
int x;  
short y = 1; // 1 is cast to short  
float z(1.5);  
char t{'@'};
```

Defining a variable allocates a region of memory large enough to store objects of the specified type and attaches the given name to that region. But the memory is not cleared in the process (again, per the standard).

Variables of the same type can be created on a single line, and this is *bad style*.

```
int x = 1, y, z = 3;
```

Variables can be used in expressions, and their values will be used in the computation:

Example 0.25

```
int main() {  
    int x = 1;  
    int y = 2;  
    std::cout << x + 3 * y << '\n'; return 0;  
}
```

As with any other entity, a variable cannot be used before it has been declared:

Example 0.26

```
int main() {  
    std::cout << x + 3 * y << '\n';  
    int x = 1;  
    int y = 2;  
    return 0;  
}
```

We get an error:

```
main.cpp: In function int main():  
main.cpp:4:16: error: x was not declared in this scope  
4 | std::cout << x + 3 * y << '\n';  
  |               ^  
main.cpp:4:24: error: y was not declared in this scope  
4 | std::cout << x + 3 * y << '\n';  
  |                   ^
```

3.3.1 Automatic Scope

Let's get acquainted with the **compound statement**.

Inside functions you can open new curly braces and list a sequence of other statements within them (which can themselves also be compound):

Example 0.27

```
int main() {  
    int x = 1;  
    { // <-- start of compound statement 1  
        int y = 2;  
        std::cout << x + 3 * y << '\n';  
        { // <-- start of compound statement 2  
            return 0;  
        } // <-- end of compound statement 2  
    } // <-- end of compound statement 1  
}
```

Compound statements are otherwise called blocks. Blocks are useful for structuring a function's code into logical parts, as well as when used together with other statements (more on this later in the course).

Important Remark. An important property of blocks is that they end the lifetime of variables declared inside the block.

Example 0.28

```
int main() {  
    int x = 1;  
    int y; // Declare y in the outer scope
```

```
{
    y = 2; // Assign a value
    std::cout << x + 3 * y << '\n';
}
return y; // y is now accessible
}
```

Thus, we find that an ordinary variable declared inside a function is valid from the point of declaration until the *nearest* end of the block in which it is declared.

Exercise 3.1 State the scope of each variable.

```
int main() {
    int x;
    {
        float y;
        {
            bool b;
            {
                char c;
            }
        }
    }
}
```

3.3.2 Global Scope

Variables can also be declared outside of functions. Such variables are called **global**.

Example 0.29

```
int x = 1;
int main() {
    int y = 2;
    std::cout << x + 3 * y << '\n';
    return 0;
}
```

Global variables exist for **the entire runtime of the program**, but can only be used after being declared!

To use a variable declared after a function, you need to declare the variable `x` inside the function and indicate that this variable is defined elsewhere. This is handled by the keyword **extern**:

Example 0.30

```
int main() {
    int y = 2;
```

```
extern int x; // declaration without a definition
std::cout << x + 3 * y << '\n';
return 0;
}
int x = 1; // definition of x
```

Important Remark. Global variables (declared outside of functions and classes) seem convenient, but in practice using them is considered an **anti-pattern** in programming. Here's why (some of the reasons):

1. **Hidden dependencies:** Any function can change a global variable, creating implicit links between parts of the program.
2. **Debugging difficulty:** When there's a bug, it's hard to tell which function changed the global state.
3. **Testing problems:** Tests depend on global state, so they can't be run in parallel or in arbitrary order.
4. **Undefined initialization order:** The order in which global variables are initialized across different files is not guaranteed.
5. **Memory waste:** Global variables live for the whole program, even if they're no longer needed. This means they take up a lot of memory.
6. **Name conflicts:** Name collisions arise, especially when using libraries.

Look at this example with global variables:

Example 0.31

```
#include <iostream>

// global variable
int global_counter = 0;

void increment() {
    global_counter++; // Who else is changing this
                      // variable?
}

void print_counter() {
    std::cout << "counter: " << global_counter <<
    std::endl;
}

int main() {
    increment();
    increment();
    global_counter += 10;
    print_counter(); // Prints 12

    // In large programs it's impossible to track
    // where else global_counter is being changed
    return 0;
}
```

If you can avoid a global variable — avoid it.. It's better to pass data explicitly through parameters or encapsulate it in objects than to create implicit dependencies through global state.

3.3.3 Storage Classes

In C++, every object in memory belongs to one of 4 classes:

- **Automatic (stack) storage.** Storage where ordinary local function variables live. They exist until the end of their block.
- **Global (static) storage.** Storage allocated at the start of the program (before main starts) and freed when it terminates.
- **Dynamic storage (the heap).** Storage is allocated and freed manually. We'll discuss this later.
- **Thread storage.** Tied to a thread of execution. Allocated when the thread starts running, freed when it finishes. Not discussed in this course.

Exercise 3.2 State the storage class, as well as the scope, of each variable.

```
int x = 0;

int main() {
    int y = 1;
    {
        extern int t;
        {
            extern int x;
            int z = 2;
        }
    }
    return 0;
}

int t = 3;
```

3.3.4 Scope

Definition 3.2. The **scope** of a variable is the part of the code from which the variable can be accessed normally by its name.

 **Important!** Scope never exceeds a variable's lifetime.

At the same time, the *scope* can be smaller than the lifetime:

Example 0.32

```
int x = 0;
int main() {
    int x = 1;
    std::cout << x << '\n'; // 1
    return 0;
}
```

In this example, the lifetime of both *x*'s extends across the *main* function. However, the outer name is hidden by the local name.

To access the global variable, you can use the **scope resolution operator** (`::`)

Example 0.33

```
int x = 0;
int main() {
    int x = 1;
    std::cout << ::x << '\n'; // 0
    return 0;
}
```

:: before a variable name tells the compiler that the name should be looked up in the global scope.

However, in the case:

Example 0.34

```
int x = 0;
int main() {
    int x = 1;
    {
        int x = 2;
        std::cout << x << '\n'; // 2
        std::cout << ::x << '\n'; // 0
    }
    return 0;
}
```

There's no way to refer to the intermediate `x` by its name.

3.3.5 Variable Initialization

Variables can be initialized at the point of definition.

Example 0.35

```
int a;           // uninitialized variable
int b = 1;       // copy initialization
int c(2);        // direct initialization
int d{3};        // uniform initialization (list initialization)
int e{};         // empty curly braces = zero-fill
```

The last 4 lines each initialize the variable's value in one way or another (write into the corresponding memory region).

To change a variable's value, you can use the assignment operator:

```
int x;
x = 10; // changed to 10
int y = 1;
y = 2; // changed to 2
```

 **Important** Initialization using `=` is **NOT** an assignment.

Reading from an uninitialized variable leads to UB. It can only be used after a specific value has been set.

3.3.6 The `const` Qualifier

Definition 3.3. A **constant object** is an object to which only operations that don't change its state can be applied (strictly speaking, there is also logical and physical constness, but more on that later).

To declare a constant variable, you need to add the word **const** to the type name:

```
const int x = 0; // the preferred form
int const y = 0;
```

Now any attempt to change `x` will result in a compile error:

```
x = 1; // CE
std::cin >> y; // CE
```

3.3.7 The volatile Qualifier

The keyword **volatile** can be appended to a variable's type. This means that operations on this variable must not be optimized away (removed, reordered relative to other operations, etc.).

This is useful when the variable is accessed not directly, but indirectly.

```
volatile int x = 0;
int volatile y = 0;
```

Important Remark. But are constants really so immutable, and is there nothing that can help us?

Let's look at an example that demonstrates how a constant can actually be changed.

Example 0.36

```
#include <iostream>
int main() {
    volatile const int x = 0;
    std::cout << x << ' ' << &x << '\n';
    int b = 10;
    int* ptr = &(b) + 1;
    std::cout << ptr << '\n';
    *ptr = 100;
    std::cout << x << ' ' << &x << '\n';
    std::cout << *ptr << ' ' << ptr << '\n';
}
```

What does this code do?

1. A **constant variable** `x` is created with the value 0, but with the **volatile** qualifier
2. The value of `x` and its address are printed
3. An ordinary variable `b = 10` is created
4. The pointer `ptr` is set to the address `b + 1` (undefined behavior! but in our case, we know it will point to the same cell as the constant, which is true in most cases)
5. The value 100 is written through this pointer
6. The value of `x` and its address are printed again, along with the value at the pointer

Why does this work (or not work)?**With `volatile`:**

- `volatile` tells the compiler: *This variable can change unexpectedly, don't optimize it*
- The compiler generates code that reads the value from memory every time
- When we change the value through an *illegal* pointer, the new value is read straight from memory

Without `volatile`:

- The compiler assumes: *since the variable is `const`, it will never change*
- It can replace every use of `x` with the constant (optimization)
- Or cache the value in a processor register
- You won't notice the change, but it happened

*What actually happens in memory?

```
// The program's stack (approximate layout):  
// Address      Value      Variable  
// 0x7fff0008    0          x (const volatile int)  
// 0x7fff0004    10         b (int)  
// 0x7fff0000    ...        something else  
  
// Pointer ptr = &b + 1 = 0x7fff0004 + 4 = 0x7fff0008  
// But in fact &b + 1 points at the variable x  
// *ptr = 100 writes at address 0x7fff0008
```

 **Important!** This is **undefined behavior (UB)**!

The code:

- Accesses memory beyond the bounds of variable `b`
- May write to a random spot on the stack
- May corrupt other variables or control structures
- In real programs will lead to unpredictable results

Conclusions

- **Don't do this in real programs!**
- `const` is a **promise to the compiler** that you won't change the value
- `volatile` is a **warning to the compiler** that the value may change from outside
- The combination `volatile const` means: *I won't change it, but someone else might*

3.4 A Bit More About Expressions

We already talked about expressions earlier (when discussing data types), but it's useful to go over this again.

3.4.1 Assignment Operators

Assignment (`=`) is used to change a variable's value:

Example 0.37

```
y = 5;  
x = y;  
x = y * y;
```

In addition to plain assignment, there are compound arithmetic assignment operators:

Example 0.38

```
x += 5; // x = x + 5  
y %= 2; // y = y % 2  
y |= 9; // z = z | 9
```

Assignment operators not only change the value but, like other operators, also return the result of their execution — the new value.

Example 0.39

```
std::cout << (y = 5); // 5  
std::cout << (x = y); // 5  
std::cout << (x = y * y); // 25
```

Example 0.40

```
std::cout << (x += 5); // 30  
std::cout << (y %= 2); // 1  
std::cout << (y |= 9); // 9
```

And this means we can build chains of assignments:

Example 0.41

```
x = y = 5;  
z = x = y *= y;
```

Moreover, it turns out that the result of an assignment operator isn't just a value, but essentially the original variable itself.

That is, you can assign something to the result again!

Example 0.42

```
(x = y) = 5; // x == 5  
(x = y) *= 0; // x == 0
```

At the same time, note that a literal or the result of a non-assigning operator cannot appear as the left operand of an assignment:

Example 0.43

```
5 = x;
```

```
x * y = 10;
```

The build fails with a strange-looking error:

```
main.cpp:8:3: error: lvalue required as left operand of assignment
8 | 5 = x;
  | ^
main.cpp:9:5: error: lvalue required as left operand of assignment
9 | x * y = 10;
  | ~^~
```

3.4.2 Result of an Expression

An expression's result manifests through

1. Its return value
2. Its side effects

Every expression is characterized by the type of its **return value** and its **value category**.

- The return value's type is straightforward (if not, go back to the section on fundamental types)
- The value category answers the question: is the result of the expression material — that is, does it exist as an object in memory?

Example 0.44

```
int x, y = 0; // variable declarations
//
-----
x + y;        // type: int    | side effect: none      |
value category: not materialized
x = 5;        // type: int    | side effect: x holds 5    |
value category: materialized (x)
x *= 2;       // type: int    | side effect: x holds 10   |
value category: materialized (x)
y + (x *= 6); // type: int    | side effect: x holds 60   |
value category: not materialized
x / 2.;       // type: double | side effect: none        |
value category: not materialized
```

3.4.3 Value Categories

The material value category is called *lvalue*, and the non-material one is called *rvalue*.

Since *lvalue* values are material (they correspond to some location in memory), they can be used as the left operand in an assignment operation (with some exceptions).

Now the nature of the earlier errors becomes clear:

```
main.cpp:8:3: error: lvalue required as left operand of assignment
8 | 5 = x;
  | ^
main.cpp:9:5: error: lvalue required as left operand of assignment
9 | x * y = 10;
  | ~^~
```

3.4.4 Increment and Decrement

Let's talk once more about increment/decrement — unary operators that increase/decrease their argument by 1.

Like assignment, they can only be applied to an *lvalue*.

Example 0.45

```
int x = 0;
x++; // postfix increment (x = 1)
++x; // prefix increment (x = 2)
x--; // postfix decrement (x = 1)
--x; // prefix decrement (x = 0)
```

If we try to increment/decrement something that is not an *lvalue*, for example:

Example 0.46

```
int x = 0;
++5;
(x + 1)--;
```

We'll get the following errors:

```
main.cpp:3:5: error: lvalue required as increment operand
3 | ++5;
main.cpp:4:6: error: lvalue required as decrement operand
4 | (x + 1)--;
```

Postfix and prefix increments/decrements differ from each other in their return value.

Prefix ++ and -- return the updated *lvalue* value (that is, the argument itself). Postfix versions return the old value (before the update) as an *rvalue*.

Example 0.47

```
int x = 0;
std::cout << ++x << '\n'; // 1 (x == 1)
std::cout << x++ << '\n'; // 1 (x == 2)
--x = 10; // x == 10
x-- = 11; // CE
```

3.4.5 The sizeof Operator

sizeof is a unary operator that returns the size of an object in bytes.

- If **sizeof** is applied to a *type*, it returns how much memory in bytes elements of that type occupy

Example 0.48

```
sizeof(char) == 1; // always
sizeof(int) == 4;  // probably (why?)
```

- If **sizeof** is applied to an expression, it computes the size of the type of the returned value.

 **Important!** **sizeof** does not evaluate the passed expression, it only analyzes it.

Example 0.49

```
long long x;
sizeof('a'); // 1
sizeof(1 / 0); // 4
sizeof(x = 1); // 8 (x is unchanged!)
sizeof(++x); // 8 (x is unchanged!)
```

3.4.6 The static_cast Operator

static_cast forcibly performs allowed implicit conversions of values from one type to another:

```
static_cast<new type> (expression)
```

Example 0.50

```
int x = 2;
int y = 2000000000;
std::cout << x / y << '\n'; // integer division
std::cout << static_cast<float>(x) / y << '\n';

std::cout << x * y << '\n'; // overflow (UB)
std::cout << static_cast<int64_t>(x) * y << '\n';
```

It is type-safe, unlike C-style type casts.

3.4.7 The Comma Operator (,)

The *comma* operator lets you combine several expressions into one:

```
++x, y--, std::cout << x + y, 5;
```

- The comma operator **,** guarantees that the expression on the left is evaluated first.
- The result of the **,** operator is the result of the second operand (the right operand).

3.5 The auto Keyword

- Before C++11, **auto** was used to denote a variable in automatic storage (a local variable).
- This feature turned out to be so irrelevant that the C++11 standard drastically changed the meaning of the word **auto** (a fairly exceptional situation).
- The old meaning was handed over to the keyword **register**.
- However, in C++17 that too was dropped, and **register** is now considered deprecated (the word is reserved for future use).

Nowadays **auto** is used for automatically deducing a variable's type during initialization

Example 0.51

```
int main() {  
    auto x = 0; // int  
    auto y = 0.0; // double  
    auto px = &x; // int*  
  
    auto z; // CE:  
}
```

Very convenient when dealing with long type names.

The rules for deducing the type of **auto** coincide with the rules for deducing template parameters (with a few exceptions).

Example 0.52

```
int x = 0; const int cx = 1; int& rx = x; int arr[10];  
  
auto y = x; // int  
auto cy = cx; // int  
auto ry = rx; // int  
auto arr_y = arr; // int*  
  
auto& z = x; // int&  
auto& cz = cx; // const int&  
auto& rz = rx; // int&  
auto& arr_z = arr; // int(&)[10]
```

 **Important!** Not everything is so wonderful with **auto**; use it carefully. We'll discuss why later.

4 Machine Arithmetic

4.1 Representing Data in a Computer

Information in a computer is stored in **binary form**. Bits are grouped into bytes (usually 8 bits each), which are the minimum addressable unit of memory.

Most modern computing machines are built on a **binary architecture**, in which information is encoded using two states: logical zero (0) and logical one (1). This representation of data is the most resistant to noise and the most convenient to implement physically using transistor circuits.

4.2 Representing Integers

Let's briefly recall data types and talk about how they're stored in memory, since this matters in the context of machine arithmetic.

4.2.1 Unsigned Integers

- Sizes: 1, 2, 4, or 8 bytes
- All bits are used to store the number's value
- Range of values:
 - 1 byte (8 bits): from 0 to $2^8 - 1 = 255$
 - 2 bytes (16 bits): from 0 to $2^{16} - 1 = 65535$
 - 4 bytes (32 bits): from 0 to $2^{32} - 1 \approx 4.29 \times 10^9$
 - 8 bytes (64 bits): from 0 to $2^{64} - 1 \approx 1.84 \times 10^{19}$

4.2.2 Signed Integers

- Sizes: 1, 2, 4, or 8 bytes
- The most significant bit is used to indicate the sign:
 - 0 — positive number
 - 1 — negative number
- Negative numbers are stored in **two's complement**.
- Range of values:
 - 1 byte: from -2^7 to $2^7 - 1$ (-128 to 127)
 - 2 bytes: from -2^{15} to $2^{15} - 1$ (-32768 to 32767)
 - 4 bytes: from -2^{31} to $2^{31} - 1$ (-2147483648 to 2147483647)
 - 8 bytes: from -2^{63} to $2^{63} - 1$ (-9223372036854775808 to 9223372036854775807)

4.3 Two's Complement

Definition 4.1. **Two's complement** is a way of representing negative integers that allows the subtraction operation to be replaced with addition.

Historical Remark. Since binary logic only has an **adder**, and no separate **subtractor** exists, there arose a need to represent negative numbers in a form such that the subtraction operation could be reduced to addition. This is how two's complement came about.

Algorithm for computing two's complement:

1. Invert every bit of the number (obtain the one's complement)
2. Add 1 to the resulting number

Example 0.53

Number	Sign-magnitude	One's complement	Two's complement
1	00000001	01111110	00000001
-1	10000001	11111110	11111111

Properties of two's complement:

- Zero has a single representation: 000...0 (which, by the way, is exactly why we add 1)
- The most negative value has no positive counterpart (for example, -128 for 8 bits)
- Addition of numbers in two's complement works correctly

Important Remark. In two's complement, one is added to the *one's complement* to eliminate the two representations of zero (+0 and -0). Positive numbers have a leading bit of 0, negative ones — 1, and the range becomes symmetric around zero (except for one extra negative number, since zero is unique).

5 Bitwise Operations in C++

Let's recall the bitwise operations we had earlier, that is, operations that work directly with the bits of numbers.

5.1 Bitwise Operators

Operator	Name	Description
&	AND	Bitwise conjunction
	OR	Bitwise disjunction
^	XOR (exclusive OR)	Bitwise exclusive OR
~	NOT	Bitwise negation (inversion)
<<	Left shift	Shift bits to the left
>>	Right shift	Shift bits to the right

 **Important!** These operators can be combined with assignment: &=, |=, ^=, <<=, >>=.

Example 0.54

```
short int a = 123; // 00000000 01111011
short int b = 321; // 00000001 01000001
short int c = a & b; // 00000000 01000001 = 65
```

Example 0.55

```
short int a = 123; // 00000000 01111011
short int b = 321; // 00000001 01000001
short int c = a | b; // 00000001 01111011 = 379
```

Example 0.56

```
short int a = 123; // 00000000 01111011
short int b = 321; // 00000001 01000001
short int c = a ^ b; // 00000001 00111010 = 314
```

Properties of XOR:

- $a \oplus a = 0$
- $a \oplus 0 = a$
- $a \oplus b = b \oplus a$ (commutativity)
- $(a \oplus b) \oplus c = a \oplus (b \oplus c)$ (associativity)

Example 0.57

```
short int a = 123; // 00000000 01111011
short int b = a << 2; // 00000001 11101100 = 492
```

For **unsigned** numbers, shifting left by n positions is equivalent to multiplying by 2^n .

Example 0.58

```
short int a = 123; // 00000000 01111011
short int c = a >> 3; // 00000000 00001111 = 15
```

For **unsigned** numbers, shifting right by n positions is equivalent to integer division by 2^n .

Important Remark. For **signed** numbers, the right shift is **arithmetic** (the sign is preserved):

```
short int a = -87; // 11111111 10101001
short int b = a << 2; // 11111110 10100100 = -348
short int c = a >> 3; // 11111111 11110101 = -11
```

 **Important!** The behavior of the right shift for negative signed numbers is compiler-dependent, but most compilers implement an arithmetic shift.

 **Important!** It's recommended to study these [tricks with bitwise operations](#).

6 Floating-Point Data Types (IEEE 754)

Let's recall that we have the following floating-point data types:

- `float` — a single-precision number (usually 4 bytes)
- `double` — a double-precision number (usually 8 bytes)
- `long double` — an extended-precision number (usually 10, 12, or 16 bytes)

And strictly speaking, the following property even holds:

$$\text{sizeof(float)} \leq \text{sizeof(double)} \leq \text{sizeof(longdouble)}$$

6.1 Representation of Floating-Point Numbers

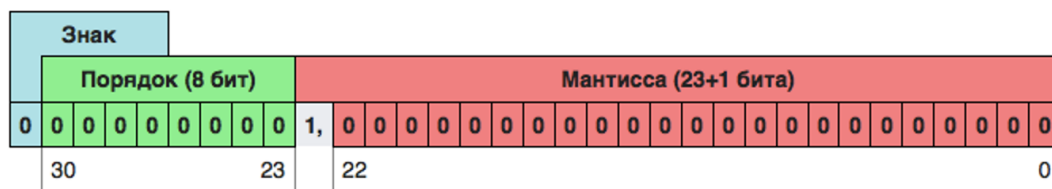
A floating-point number is represented as:

$$x = (-1)^{\text{Sign}} \times (1.M) \times 2^{P-S}$$

where:

- **Sign** — the sign of the number (0 — positive, 1 — negative)
- **M** — the mantissa (fractional part of the normalized number)
- **P** — the exponent in its binary representation
- **S** — the exponent bias, equal to $2^{k-1} - 1$, where k is the number of bits allocated to the exponent

The representation's structure looks like this (here we're using the 32-bit `float` type).



Before encoding, the number is normalized: it's brought to the form $1.M \times 2^E$, where:

- $1.M$ — the mantissa with an *implicit leading one* (not stored in memory, because there's no point — in binary representation it's always a one there)
- E — the true exponent ($E = P - S$). We need the bias so that we don't have to work with negative numbers in the exponent.

Example 0.59

Let's represent the number $A = -121.1_{10}$ the way it's stored in memory.

1. Conversion to binary:

$$-121_{10} = -1111001_2$$

$$0.1_{10} = 0.0(0011)_2 \quad (\text{an infinite repeating fraction})$$

$$A = -1111001.00011_2$$

2. Normalization:

$$A = -1.1110010(0011)_2 \times 2^6$$

$$M = 1110010(0011)_2 \quad (\text{fractional part of the mantissa})$$

$$E = 6 \quad (\text{true exponent})$$

$$S = 2^{8-1} - 1 = 127 \quad (\text{bias for an 8-bit exponent})$$

$$P_{\text{biased}} = E + S = 6 + 127 = 133_{10} = 10000101_2$$

3. In total:

- Sign: 1 (negative number)
- Exponent: 10000101
- Mantissa: the first 23 bits of 1110010(0011):
11100100011001100110011 (the remaining bits are rounded)

4. Final representation (32 bits):

1 10000101 11100100011001100110011

Or in hexadecimal: 0xC3F23333

Important Remark. So why is $0.1 + 0.2 \neq 0.3$ in binary? The decimal fraction 0.1_{10} cannot be represented exactly in binary, since it's an infinite repeating fraction in binary:

$$0.1_{10} = 0.0(0011)_2 = 0.00011001100110011 \dots_2$$

When storing it in the IEEE 754 format (32 or 64 bits), the following can occur:

- **Rounding.** The infinite fraction is cut off at 23 (or 52) mantissa bits
- **Representation errors.** Since an approximate value is stored.
- **Error accumulation,** i.e. when adding two imprecise values, the error accumulates.

$$0.1_{10} \approx 1.10011001100110011001101 \times 2^{-4}$$

$$0.2_{10} \approx 1.10011001100110011001101 \times 2^{-3}$$

$$0.1 + 0.2 \approx 0.3000000000000000444089 \dots \neq 0.3$$



this is a limitation of positional number systems and an architectural quirk, not a bug in the computer or the language. For precise computations with decimal fractions, special data types are used.

6.2 Loss of Significance

Subtracting close floating-point numbers loses precision:

Example 0.60

```
float a = 1000000.0f;
```

```
float b = 1000000.1f;
float c = b - a; // we expect 0.1, but get an inexact value
```

6.3 Non-Associativity of Addition

For floating-point numbers, addition is not associative because of rounding:

Example 0.61

```
float a = 1e30f;
float b = -1e30f;
float c = 1.0f;

float r1 = (a + b) + c; // 1.0
float r2 = a + (b + c); // 0.0 (due to loss of precision)
```

 **Important!** Unlike mathematics, where addition is associative $(a + b) + c = a + (b + c)$, in floating-point computer arithmetic this property doesn't hold, due to the finite precision of the representation and the rounding rules.

7 Quirks of Working with Large Numbers

Besides the quirks of working with floating-point numbers, architectural limitations also give us quirks with integers, such as overflow.

Overflow occurs when the result of an arithmetic operation exceeds the maximum value that can be represented in that data type.

Important Remark. Overflow of signed and unsigned numbers:

- **Unsigned numbers:** wraps around cyclically
 $255 + 1 = 0$ for unsigned `char` (8 bits)
- **Signed numbers:** behavior is undefined in C/C++, but at the processor level a cyclic wraparound usually happens:
 $127 + 1 = -128$ for signed `char`, for example. (8 bits, two's complement)

Example 0.62

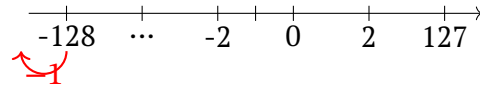
```
unsigned char u = 255; // maximum unsigned value
u++;                  // u = 0 (cyclic wraparound)

signed char s = 127;  // maximum positive signed value
s++;                  // Undefined behavior in C/C++! In
                      // practice, often s = -128

signed char t = -128; // minimum signed value
t--;                  // Undefined behavior! In practice,
```

```
often t = 127
```

Cyclic wraparound on overflow



Глава 1

Seminars

1 Getting to Know C++. Data Types and Basic Operators.

Question 1.1 Which of the following strings are valid identifiers in C++? Explain why the others aren't.

1. my_var
2. _count
3. 2fast
4. **for**
5. user-name
6. User_Name

Question 1.2 Determine the type of each of the following literals.

1. 42
2. 42U
3. 42L
4. 42LL
5. 3.14
6. 3.14f
7. **'A'**
8. **"A"**

Question 1.3 Determine the result of the expression:

`(5 > 3) && (2 == 0)`

Important Remark. The `&&` operator (logical AND) evaluates its right operand only if the left one is **true**. This is the so-called **short-circuit evaluation** — the behavior of logical operators whereby the right operand is only evaluated if it's actually needed to determine the result of the whole expression.

For the `&&` operator (logical AND): if the left operand is **false**, the result of the whole expression is guaranteed to be **false**, so there's no point evaluating it. The same thing happens with logical OR (`||`): if the left operand is **true**, the result is guaranteed to be **true** and the right operand is not evaluated.

Question 1.4 Explain the difference between `&` and `&&`, and between `|` and `||`. Give examples.

Question 1.5 What is the type of the result of the expression `'a' + 1`?

Important Remark. In C++, when performing arithmetic operations on the **char** type, it is first converted to **int**. Roughly speaking, you can remember it this way: all integer types smaller than **int** are converted to **int** during arithmetic operations.

Question 1.6 What is the result of the expression? Explain the order of evaluation:

```
int x = 5 + 3 * 2 - 4 / 2;
```

Comment. The order of evaluation is determined by operator precedence: multiplication and division have higher precedence than addition and subtraction.

Question 1.7 Consider the following code and explain the result:

```
#include <iostream>

int main() {
    int a = 0;
    std::cout << a << ' ' << a++ << a << '\n';
    std::cout << a << ' ' << ++a << a << '\n';
    std::cout << a << ' ' << a-- << a << '\n';
    std::cout << a << ' ' << --a << a << '\n';
    return 0;
}
```

Comment. Here, `a++` (postfix increment) returns the old value and then increases `a` by 1, whereas `++a` (prefix increment) increases `a` by 1 and then returns the new value.

Important Remark. In most cases, prefix increment (`++x`) is preferable to postfix (`x++`), especially for non-trivial types. **Prefix** (`++x`) increment first increases the value and then returns the new value. **Postfix** (`x++`), on the other hand, first saves the old value, increases the value, and only then returns the old value. That's clearly an extra operation and extra memory cost. For built-in types (`int`, `char`, etc.) there's actually NO difference. Modern compilers optimize both variants the same way. But for user-defined types (classes) there is a difference! So use postfix increment only when you genuinely need the old value. For all other cases, there's **prefix increment**.

Important Remark. Actually, strictly speaking, this code involves Undefined Behavior. **The order in which arguments are evaluated in a function call is not defined by the standard.** This means the compiler can evaluate the operands of << in any order.

Why is this UB? The C++ standard does not define the order in which the operands of << are evaluated. The compiler can:

- Evaluate all the arguments left to right
- Evaluate all the arguments right to left
- Evaluate them in an arbitrary order

What might the output be? The result depends on the compiler, the compiler version, and even the optimization level. For example:

- GCC might print one thing
- Clang might print something else
- With the -O0 and -O3 flags the result may differ

Avoid modifying the same variable multiple times within a single expression. Split such operations into separate lines:

```
int a = 0;
std::cout << a << ' ';
std::cout << a++ << ' ';
std::cout << a << '\n';
```

Don't rely on any particular evaluation order in expressions with side effects!

Question 1.8 Consider the following code and explain the result:

```
#include <iostream>

int main() {
    int a = 0;
    std::cout << (a++ + ++a);
    return 0;
}
```

Question 1.9 Determine the value of the variable y after the following code runs:

```
int main() {
    int x = 5;
    int y = (x = 10) + (x = 20);
    return y;
}
```

Question 1.10 What will the following program print?

```
#include <iostream>

int main() {
    unsigned char x = 255;
```

```
x++;  
std::cout << (int)x << "\n";  
return 0;  
}
```

Important Remark. The result is striking — this is overflow.

Question 1.11 What happens when large numbers are added? Why?

```
#include <iostream>  
  
int main() {  
    int a = 1000000000; // 1 billion  
    int b = 2000000000; // 2 billion  
    int c = a + b;  
    std::cout << c;  
    return 0;  
}
```

Question 1.12 What happens in the following cases?

1. `int a = INT_MAX; a++;`
2. `int b = -1; b >> 1;`
3. `int c = 5 / 0;`

Question 1.13 What will the result be? Why?

```
#include <iostream>  
  
int main() {  
    int a = 20, b = 50;  
    std::cout << a / b << '\n';  
    return 0;  
}
```

Question 1.14 What will the program print?

```
#include <iostream>  
  
int main() {  
    int a = 5;  
    int b = (a *= 2, a + 3);  
    std::cout << "a=" << a << ", b=" << b;  
    return 0;  
}
```

Comment.

Definition 1.1. The comma operator ‘,’ is a binary operator that:

1. Evaluates the left operand
2. Discards its result (unless it's **void**)
3. Evaluates the right operand
4. Returns the result of the right operand

Importantly, the comma operator ‘,’ provides a **guaranteed order of evaluation**. But at the same time it has the **lowest precedence** of all C++ operators!

Question 1.15 What will the program print?

```
int main() {  
    int a = 1, b = 2, c = 3;           // 1  
    int x = (a++, b++, c++);           // 2  
    std::cout << a << b << c;         // 3  
    int y = (a, b, c);                 // 4  
    return (x = 5, y = 10, x + y);     // 5  
}
```

Exercise 1.1 Analyze how boolean types work.

```
#include <iostream>  
  
int main() {  
    bool a = true;  
    std::cout << a << '\n';           // Prints 1  
    bool b = false;  
    std::cout << b << '\n';           // Prints 0  
    std::cout << (5 > 3) << '\n';     // Prints 1  
    std::cout << (2 == 0) << '\n';    // Prints 0  
  
    return 0;  
}
```

Comment. The result is amusing — instead of **true** and **false** you get 0 or 1. In C++, the boolean type **bool** is an integer type, where **true** corresponds to 1 and **false** corresponds to 0. By default, the << operator for the type **bool** prints its numeric representation (0 or 1), not the strings "true"/"false". This behavior of printing 0/1 instead of true/false was inherited from the C language, which didn't have a **bool** type before the C99 standard. In C, boolean values were represented as **int**, where 0 = false and any nonzero value = true. C++ preserved this compatibility. By the way, C++ has implicit conversions between **bool** and integer types.

Important Remark. To print the strings "true"/"false" use the `std::boolalpha` manipulator.

```
#include <iostream>

int main() {
    bool a = true;
    std::cout << std::boolalpha << a << std::noboolalpha
    << a;
    return 0;
}
```

Exercise 1.2 Write a program that prints the sizes (in bytes) of the following types on the current platform:

- `char`, `short`, `int`, `long`, `long long`
- `float`, `double`, `long double`
- `bool`

Solution Outline. Use the `sizeof` operator to determine the size of each type.

Important Remark. A type's size can vary across different platforms and compilers. As we said in the lecture, the standard only guarantees a minimum size, but doesn't strictly specify the exact size for some types. In cases where this matters, use fixed-width types.

Exercise 1.3 Include the header `<cstdint>` and work with fixed-width types:

1. Declare a variable of an 8-bit signed integer type
2. Declare a variable of a 32-bit unsigned integer type
3. Assign their sum to another variable and print it. Think about what type this sum should be.

Exercise 1.4 Write a program that prints the number `10.2427` with three digits after the decimal point.

Solution Outline. For formatted output, use the `std::fixed` and `std::setprecision` manipulators from the `<iomanip>` library.

2 Memory and Data Types. Storing Numbers in Memory. Quirks of Converting and Subtracting Large Numbers. Bitwise Operators.

Exercise 2.1 Write a program that takes two integers a and b and prints:

1. The quotient of a / b (integer division)
2. The remainder of $a \% b$
3. Checks whether $a == (a / b) * b + (a \% b)$ actually holds

Important Remark. Division by zero leads to undefined behavior (Undefined Behavior). Try running the code several times and look at the result.

Exercise 2.2 Given the number $x = 9$. Using only bitwise operators:

1. Set the second bit (counting from zero) to 1
2. Clear the third bit
3. Check whether the number is even

Important Remark. Bitwise operations are often faster than arithmetic ones and are sometimes used in compression and hashing algorithms.

Exercise 2.3 Write a program that:

1. Reads a single character from the keyboard
2. If it's an uppercase Latin letter, converts it to lowercase

Solution Outline. Use the `std::cin` stream from the `<iostream>` library for input. And for type conversion, use the C-style cast `double b = (double)a;`. Although its use is considered outdated and unsafe, we'll still use it for now, but we'll fix that later.

Exercise 2.4 Write a program that prints the ASCII codes of all the digits from `'0'` to `'9'`.

Question 2.1 What will the program print?

```
#include <iostream>
int main() {
    std::cout << 11 / 4 << '\n';
}
```

Question 2.2 What will the program print?

```
#include <iostream>
int main() {
```

```
std::cout << 11.0 / 4 << '\n';  
}
```

Question 2.3 What will the program print?

```
#include <iostream>  
int main() {  
    int a = 0;  
    bool b = true || (++a > 0);  
    std::cout << a << '\n';  
}
```

Question 2.4 What will the program print?

```
#include <iostream>  
int main() {  
    int a = 0;  
    bool b = false && (++a > 0);  
    std::cout << a << '\n';  
}
```

Question 2.5 What will the program print?

```
#include <iostream>  
int main() {  
    unsigned short a = 6, b = 3;  
    std::cout << (a & b) << " " << (a | b) << " " << (a ^ b) <<  
    '\n';  
}
```

Question 2.6 What will the program print?

```
#include <iostream>  
int main() {  
    unsigned short a = 3;  
    std::cout << (a << 3) << " " << (a >> 1) << '\n';  
}
```

Question 2.7 What will the program print?

```
#include <iostream>  
int main() {  
    short a = -8;  
    std::cout << (a >> 1) << '\n';  
}
```

Important Remark. A right shift of signed integers is implementation-defined (it depends on the compiler's implementation) — most compilers perform an arithmetic shift (the sign is preserved).

Question 2.8 What will the program print?

```
#include <iostream>
int main() {
    std::cout << sizeof(0) << " " << sizeof(0L) << " " <<
    sizeof(0LL) << '\n';
}
```

Question 2.9 What will the program print?

```
#include <iostream>
int main() {
    std::cout << (0u + 0) << " " << (0ul + 0ll) << '\n';
}
```

Question 2.10 What will the program print?

```
#include <iostream>
int main() {
    double a = 0.1, b = 0.2;
    std::cout.setf(std::ios::fixed);
    std::cout.precision(17);
    std::cout << (a + b) << '\n';
}
```

Question 2.11 What will the program print?

```
#include <iostream>
int main() {
    double x = 1.0/0.0;
    double y = 0.0/0.0;
    std::cout << (x > 0) << " " << (x == x) << " " << (y == y)
    << '\n';
}
```

Question 2.12 What will the program print?

```
#include <iostream>
int main() {
    unsigned int a = 12, b = 5;
    std::cout << (a & b) << " " << (a | b) << " " << (a ^ b) <<
    '\n';
}
```

Question 2.13 What will the program print?

```
#include <iostream>
int main() {
    unsigned int x = 7;
    std::cout << ~x << '\n';
}
```

Important Remark. The result depends on the size of `unsigned int` on the current platform.

Question 2.14 What will the program print?

```
#include <iostream>
int main() {
    unsigned int x = 3;
    std::cout << (x << 2) << '\n';
}
```

Comment. A left shift by n bits is equivalent to multiplying by 2^n .

Question 2.15 What will the program print?

```
#include <iostream>
int main() {
    unsigned int x = 32;
    std::cout << (x >> 3) << '\n';
}
```

Comment. A right shift by n bits is equivalent to integer division by 2^n .

Question 2.16 What will the program print?

```
#include <iostream>
int main() {
    unsigned int x = 5;
    std::cout << ((x << 1) + (x >> 1)) << '\n';
}
```

Question 2.17 What will the program print?

```
#include <iostream>
int main() {
    unsigned int a = 42;
    std::cout << (a ^ a) << '\n';
}
```

Question 2.18 What will the program print?

```
#include <iostream>
int main() {
    unsigned int a = 7, b = 12;
    a = a ^ b;
    b = a ^ b;
    a = a ^ b;
    std::cout << a << " " << b << '\n';
}
```

```
}
```

Question 2.19 What will the program print?

```
#include <iostream>
int main() {
    unsigned int mask = 1 << 3; // 8
    unsigned int x = 13;
    std::cout << (x & mask) << '\n';
}
```

Question 2.20 What will the program print?

```
#include <iostream>
int main() {
    unsigned int x = 5;
    unsigned int mask = 1 << 2;
    std::cout << (x | mask) << '\n';
}
```

Question 2.21 What will the program print?

```
#include <iostream>
int main() {
    unsigned int x = 13;
    unsigned int mask = ~(1 << 2);
    std::cout << (x & mask) << '\n';
}
```

Question 2.22 What will the program print?

```
#include <iostream>
int main() {
    unsigned int x = 9;
    unsigned int mask = 1 << 3;
    std::cout << (x ^ mask) << '\n';
}
```

Question 2.23 What will the program print?

```
#include <iostream>
int main() {
    unsigned int x = 57;
    std::cout << (x & 1) << '\n';
}
```

Question 2.24 What will the program print?

```
#include <iostream>
int main() {
    unsigned int x = 11;
```

```
std::cout << (x << 1) << '\n';  
}
```

Question 2.25 What will the program print?

```
#include <iostream>  
int main() {  
    unsigned int x = 25;  
    std::cout << (x >> 1) << '\n';  
}
```

Question 2.26 What will the program print?

```
#include <iostream>  
int main() {  
    unsigned int x = 6;  
    unsigned int y = 10;  
    std::cout << ((x & y) << 1) + (x | y) << '\n';  
}
```

Comment. This is a fairly interesting bitwise formula: $((x \& y) \ll 1) + (x | y)$. Try to figure out what it computes.

Exercise 2.5 Write a program that:

1. Takes a four-bit decimal number (from 0 to 15)
2. Reverses its bits. This means mirroring the order of all the bits in its binary representation, using the minimum required length
3. Prints the original and transformed number in various formats

Important Remark.

Here's a fun fact.

- **Big-endian (most significant to least significant)** — a notation in which the most significant byte is stored at the lowest address.
- **Little-endian (least significant to most significant)** — a notation in which the least significant byte is stored at the lowest address.

Strictly speaking, this is just the order in which elements are numbered and stored in memory. **Endianness** is the way multi-byte data (integers, floating-point numbers, etc.) is stored in a computer's memory or transmitted over a network.

Exercise 2.6 Write a program that swaps the values of two integers using only bitwise operations.

Exercise 2.7 Write a program that compares the execution time of two operations:

1. Dividing by 2 using the / operator
2. Dividing by 2 using a bitwise right shift >>

Important Remark. We'll use <chrono> to measure the time.

Basic template for timing measurements:

```
auto start = std::chrono::high_resolution_clock::now();

// Code to measure (there will be loops here)
for (int i = 0; i < 100000; ++i) {
    // operation
}

auto end = std::chrono::high_resolution_clock::now();
auto duration = std::chrono::duration<double,
std::milli>(end - start);
std::cout << "Time: " << duration.count() << " ms\n";
```

Why exactly 100000 operations?

- Too few operations (e.g., 10-100) — the time is too short to measure accurately
- Too many operations (e.g., 100 million) — the test will take too long to run

There are various units of measurement you can use:

```
// Milliseconds
auto ms = std::chrono::duration<double,
std::milli>(end - start);

// Microseconds
auto us = std::chrono::duration<double,
std::micro>(end - start);

// Nanoseconds
auto ns = std::chrono::duration<double, std::nano>(end
- start);

// Seconds
auto sec = std::chrono::duration<double>(end - start);
```

Historical Remark. In the past, shifting was significantly faster than division (by a factor of 10-100), since division is a complex operation for the processor. Modern processors have optimized division units, and compilers are good at optimizing division by a power of two. However, for embedded systems or when working in assembly, preferring shifts is still relevant.

Exercise 2.8 Write a program that demonstrates working with various math functions: take an absolute value, raise to a power, take a root of a number, and also print out some constants.

Important Remark. Main functions of the <cmath> library:

- $\sin(x)$, $\cos(x)$, $\tan(x)$ - sine, cosine, tangent (in radians)
- $\asin(x)$, $\acos(x)$, $\atan(x)$ - arcsine, arccosine, arctangent
- $\atan2(y, x)$ - the arctangent of y/x (accounts for the quadrant)
- $\exp(x)$ - the exponential function (e^x)
- $\log(x)$ - the natural logarithm ($\ln x$)
- $\log_{10}(x)$ - the base-10 logarithm ($\log_{10} x$)
- $\log_2(x)$ - the base-2 logarithm ($\log_2 x$)
- $\text{pow}(x, y)$ - raising to a power (x^y)
- $\text{sqrt}(x)$ - square root ($\sqrt{x}$)
- $\text{cbrt}(x)$ - cube root ($\sqrt[3]{x}$)
- $\text{ceil}(x)$ - round up ($\lceil x \rceil$)
- $\text{floor}(x)$ - round down ($\lfloor x \rfloor$)
- $\text{round}(x)$ - nearest integer
- $\text{trunc}(x)$ - discard the fractional part
- $\text{fmod}(x, y)$ - remainder for float/double division
- $\text{remainder}(x, y)$ - remainder rounded to the nearest value

Constants:

- `std::numbers::pi` - the number π
- `std::numbers::e` - Euler's number e
- Require the header `<numbers>`

Exercise 2.9 Write a program that generates two random numbers.

Important Remark. For working with random numbers, it's better to use the **<random> library** with some generator:

- `std::mt19937` - Mersenne Twister
- `std::mt19937_64` - the 64-bit version
- `std::minstd_rand` - a linear congruential generator
- `std::ranlux48` - high quality, but slow

There are also various **distributions**

- `std::uniform_int_distribution<int>` - uniform integer
- `std::uniform_real_distribution<double>` - uniform real
- `std::normal_distribution<double>` - normal (Gaussian)
- `std::bernoulli_distribution` - Bernoulli
- `std::binomial_distribution<int>` - binomial
- `std::poisson_distribution<int>` - Poisson
- `std::exponential_distribution<double>` - exponential

```
#include <random>
#include <iostream>

int main() {
    // Initialize the generator
    std::random_device rd;
    std::mt19937 gen(rd());

    // Uniform distribution of integers from 1 to 100
    std::uniform_int_distribution<int> dist_int(1, 100);

    // Uniform distribution of reals from 0.0 to 1.0
    std::uniform_real_distribution<double> dist_real(0.0,
1.0);

    // actually generate the numbers
    for (int i = 0; i < 10; ++i) {
        std::cout << "Integer: " << dist_int(gen) << "\t"
        << "Real: " << dist_real(gen) << "\t"
        << "Normal: " << dist_normal(gen) << '\n';
    }

    return 0;
}
```